WEB 250 asgn08 debugger setup

Student review handout 1 of 5 | Ungraded practice

The goal is to pause your working Sandbox, watch its variables change, and follow one method call at a time. Start with the same local project and database connection that already display your birds.

1 Open the project that the web server is using

1. Start Apache and MySQL in MAMP or XAMPP.

2. In VS Code, use **File → Open Folder** and open your actual asgn08 folder.

3. Open the Sandbox through its existing http://localhost... address and confirm that it works.

Common project locations are /Applications/MAMP/htdocs/web250/asgn08 on Mac and C:\xampp\htdocs\web250\asgn08 on Windows. Use your actual document root if you changed it.

The class's earlier debugger guide used Apache port **55000**. With that setup, the Sandbox URL is:

```
http://localhost:55000/web250/asgn08/public/sandbox_active_record.php
```

Keep a different working Apache port if that is what your installation uses. The three ports have different jobs:

Port	Job
Your Apache port, such as 55000 or 8888	Browser requests your PHP page.
Your existing MySQL port	PHP connects to your database.
9003	Xdebug communicates with VS Code.

Use the browser to request the page while debugging. The VS Code listener waits for that request; clicking its green triangle does not open the page for you. [1]

2 Check the PHP configuration used by the browser

Create a temporary file named public/debug_info.php:

```
<?php
phpinfo();
```

Open this file with the same localhost base URL as the Sandbox. PHP displays its configuration. Record the **PHP Version** and **Loaded Configuration File** near the top. Search the page for an **Xdebug** section. [2]

The browser's configuration is the one that matters for this activity. A terminal command such as php -v can use a different PHP installation; PHP can also select different configuration files for different ways of running it. [3]

If Xdebug 3 is already listed, Mac users should also check the FastCGI paragraph in Section 3A, then continue to Section 4. If it is absent, use the section for your operating system below.

3A Mac enable Xdebug in MAMP

1. Stop the MAMP servers and save a backup copy of the active php.ini before editing it.

2. Open the **Loaded Configuration File** you just recorded. For regular MAMP it commonly has this pattern:

```
/Applications/MAMP/bin/php/phpX.Y.Z/conf/php.ini
```

X.Y.Z stands for the PHP version selected in your installation. MAMP has separate configuration files for its PHP versions. [4]

3. Search this file for xdebug. If it contains a commented zend_extension line pointing to that version's xdebug.so, check that the file exists, then remove the initial semicolon to enable it. Keep the actual path supplied with your MAMP installation.

4. If there is no existing line, inspect the active PHP version's lib/php/extensions folder for xdebug.so. When that matching file exists, add a zend_extension line containing its full path. The following is a pattern, **not a path to copy unchanged**:

```
zend_extension="/full/path/to/the/matching/xdebug.so"
```

Xdebug must load as a zend_extension; use the binary that matches the PHP installation you are running. [5] If that MAMP PHP version has no matching Xdebug file, ask the instructor to help select/install a compatible MAMP PHP/Xdebug combination. Do not borrow an extension from a different PHP version merely because its filename matches.

Then complete Section 4 before restarting.

MAMP FastCGI adjustment for classroom pauses

This is the adjustment that resolved the instructor's MAMP debugger timeout during the earlier class setup. It gives the paused request more time; it does not install or activate Xdebug.

With the MAMP servers stopped, make a backup copy of:

```
/Applications/MAMP/conf/apache/httpd.conf
```

Find the existing line:

```
FastCgiServer /Applications/MAMP/fcgi-bin/php.fcgi -socket httpdFastCGI.sock
```

Add -idle-timeout 600 to that same line. **Keep the directive on one physical line.** It may wrap visually in this handout. The student ZIP includes a copyable version in debugger_files/mamp-fastcgi-line.txt:

```
FastCgiServer /Applications/MAMP/fcgi-bin/php.fcgi -socket httpdFastCGI.sock -idle-timeout 600
```

Keep other existing options. If the line already includes -idle-timeout, adjust its value instead of adding the option a second time. Save, then restart the servers after finishing the PHP settings below. If your configuration does not contain this FastCgiServer entry, have the instructor check it; do not insert a duplicate or unrelated server declaration. Windows/XAMPP users skip this adjustment.

3B Windows enable Xdebug in XAMPP

1. Use the official Xdebug installation wizard.

2. Copy the complete output of the **browser's** debug_info.php page into the wizard. It provides instructions for the PHP build that produced that output. [6]

3. Follow its download instructions. Put the matching DLL in the PHP extension folder it identifies, commonly C:\xampp\php\ext.

4. Stop Apache. Save a backup copy of the browser's **Loaded Configuration File**, commonly C:\xampp\php\php.ini, then edit it.

5. Add or enable the zend_extension entry using the exact DLL name/path given by the wizard. Do not add a second entry if one already loads Xdebug. [5]

Then complete Section 4 and restart Apache.

4 Configure Xdebug 3 for local practice

In the same php.ini, set these values. Update existing entries if present:

```
xdebug.mode = debug
xdebug.start_with_request = yes
xdebug.client_host = 127.0.0.1
xdebug.client_port = 9003
```

These settings enable step debugging, start it for each PHP request, and direct the debugger connection to VS Code on your own computer. 9003 is Xdebug 3's default debugging port. [7]

Save the file and restart the web server. Reload debug_info.php in the browser. Confirm that it has an Xdebug 3 section and that the four values above are active. This check separates “the file was edited” from “the running PHP process loaded the settings.”

5 Set up the VS Code listener

Install the **PHP Debug** extension published by **Xdebug** (xdebug.php-debug). In the asgn08 folder, create a .vscode folder and a launch.json file inside it. If you already have this file, add the configuration to its existing configurations array.

For a new file, use:

```
{
   "version": "0.2.0",
   "configurations": [
      {
         "name": "Listen for Xdebug - asgn08",
         "type": "php",
         "request": "launch",
         "port": 9003
      }
   ]
}
```

This listener uses the PHP Debug adapter and the same port as Xdebug. Select it in the **Run and Debug** panel. [1]

6 Prove that a breakpoint works

1. Open public/sandbox_active_record.php in VS Code.

2. Click the margin beside its first require_once statement to place a breakpoint on that executable line.

3. In **Run and Debug**, select **Listen for Xdebug - asgn08** and click the green triangle.

4. Refresh sandbox_active_record.php in the browser.

5. Return to VS Code. Execution should be paused with the current line highlighted.

A breakpoint stops before the highlighted statement runs. A variable assigned on that line will show its new value after you execute the line. It is normal for the browser to wait while PHP is paused.

Use **Continue** to finish the request. The Sandbox should display its birds. Now use handout 3, Sandbox breakpoints, to follow the actual method calls.

7 The controls we will use

Hover over toolbar buttons to see their names. On a Mac, you may need the **Fn** key for the function-key shortcuts; clicking the buttons works too.

Control	Shortcut	Use during this review
Continue	F5	Run until another breakpoint or the end of the request.
Step Over	F10	Execute the highlighted line without entering its method call.
Step Into	F11	Enter a PHP function or method called on the highlighted line.
Step Out	Shift+F11	Finish this function/method and return to its caller.
Stop	Shift+F5	End the debugging session.

Inspect values in **Variables** or by hovering over expressions. **Call Stack** shows the active calls that led to the current line. Selecting a stack frame changes which function's local variables you are viewing. [8]

PHP's built-in PDO methods do not have student PHP source to step through. Use Step Over on calls such as query(), prepare(), execute(), and fetchAll(), then inspect their results. Step Into is most useful for your own Database, SandboxBird, and Bird methods.

8 If the page does not pause

What you observe	What to check next
No Xdebug section in the browser's information page	The browser's PHP did not load Xdebug. Recheck its actual configuration file and extension path.
Xdebug appears, but nothing pauses	Confirm VS Code is listening, the port is 9003 on both sides, and xdebug.mode / xdebug.start_with_request have the Section 4 values.
The wrong file opens or your breakpoint is never reached	Make sure VS Code has the same asgn08 directory that the localhost URL serves; check for a second copy.

What you observe	What to check next
A breakpoint stays unverified during the request	Put it on an executable PHP statement, such as require_once, rather than a comment or blank line.
The browser keeps loading	Look at VS Code. PHP may simply be paused and waiting for Continue.
MAMP fails after a long pause	Recheck the FastCGI timeout adjustment and restart the servers.
“Address already in use” on 9003	Stop a second debugger listener, including one in another VS Code window.

When the review is finished, delete public/debug_info.php before staging your work in Git. If you keep it during local testing, add public/debug_info.php to your asgn08 .gitignore. Keep configuration inspection local; this temporary file is not part of the deployed assignment.

References

1. Xdebug — PHP Debug adapter for VS Code: installation, listener configuration, and browser request behavior.
2. PHP Manual — phpinfo(): configuration and extension information.
3. PHP Manual — The configuration file: configuration selection and when PHP reads it.
4. MAMP — Where can I find php.ini?: version-specific MAMP paths.
5. Xdebug — Installation: extension loading and Windows installation.
6. Xdebug — Installation wizard: tailored download/configuration instructions.
7. Xdebug — Step Debugging: debug mode, request activation, host, and port settings.
8. VS Code — Debug code: controls, breakpoints, variables, and call stack.

The MAMP FastCGI line comes from the instructor's previously confirmed working course setup.